Attendee Workbook — Genie & Governed Metrics

Welcome. This is the **Genie track** of Analyst 101: you'll build a **Genie Agent** on the shared
dataset, tune it so plain-language questions resolve to the right answer (Part 1), then add the
accuracy layer — SQL functions, metric views, and trusted assets — and prove the lift with a
benchmark (Parts 2–5). Follow along on your own screen, and ask questions any time. All data is
synthetic — **no PHI**.

The dataset

Everything lives in `{{CATALOG}}.{{SCHEMA}}`:

Table	What it is
`fact_encounters`	One row per patient encounter. ~80,000 rows over Jan 2023 to Dec 2025. Outcome flags: `readmitted_30d`, `mortality_flag`, `complication_flag`.
`dim_provider`	Providers, their specialty, and primary facility.
`dim_facility`	Hospitals and clinics, with type, city, state, region.
`dim_diagnosis`	ICD-10 codes with plain-language descriptions and clinical category.
`dim_procedure`	Procedure codes (incl. `{{PROCEDURE_CODE}}` {{PROCEDURE_EXAMPLE}}).
`dim_patient`	Patients with their assigned PCP and home facility (used for the PCP-continuity metric).
`dim_visit_type`	Visit type reference; non-standard types are excluded from PCP continuity.

Part 1 - Build a Genie Agent

Now the fun part. Here you'll build a full **Genie Agent** (this is what used to be called a "Genie space") on the shared dataset, give it a few instructions, and ask questions in

plain language. *(A published AI/BI dashboard also has an Ask Genie box for quick questions on just that dashboard's data — the Genie Agent you build here is the full, reusable version across the whole dataset.)*

Step 1 · Create a Genie Agent — on the shared tables.

- In the left sidebar, click **Genie Agents** → **New** (top-right). *(If your workspace still says "Genie" / "Genie spaces," it's the same thing.)*
- Add these **data sources** (click **+ Add data** / the table picker), all from the **shared** schema `{{CATALOG}}.{{SCHEMA}}`: `fact_encounters`, `dim_provider`, `dim_facility`, `dim_diagnosis`, `dim_procedure`.
- Click **Create**. **Genie Code** launches automatically — that's where you add instructions and trusted assets.

Step 2 · Add instructions.

- In **Genie Code** (the agent's **Instructions** area), paste the block below (your instructor also shares it). Save.

Instructions block to paste (scope + conventions only — no join prose: relationships and the example query carry the joins):

```
This Genie Agent answers questions about synthetic hospital encounters. There is no PHI.

Conventions:
- Express all rates as a percentage from 0 to 100, rounded to one decimal; currency as wh
- When ranking providers or facilities by a rate, only include those with at least 200 en
  unless the user asks otherwise.
- Show plain-language names in results (provider_name, facility_name, region, clinical_ca
  procedure_description), not the id columns.
```

> *Why no "data model / joins" here:* the shared tables' joins are already declared as foreign keys — so restating joins in prose is redundant. Text instructions are a last resort; relationships and examples are more reliable. If a join ever misses, define it as an explicit **relationship** in Genie Code, not in text.

Step 3 · Add SQL-based context — this is what really tunes Genie. Free text is the *last* resort; **SQL-based context is more reliable**. Add a few high-value pieces in Genie Code (the same levers Databricks Day goes deep on):

- **Synonyms** — map everyday words to a **column** (synonyms attach to columns, not tables). In Genie Code → the data source → **Edit column metadata** → pick the column → **Synonyms** field. Add each of these:
- `dim_provider.provider_name` ← **doctor**, **physician**
- `dim_facility.facility_name` ← **hospital**, **clinic**, **site**
- `fact_encounters.length_of_stay_days` ← **LOS**, **length of stay**

- `fact_encounters.readmitted_30d` ← **readmission**, **readmit**, **bounce-back**
- `fact_encounters.total_paid` ← **reimbursement**, **amount paid**

Now *"which doctors have the most encounters?"* and *"average LOS by hospital"* resolve without anyone knowing the column names.
- **SQL expressions** — reusable, named metrics/filters. In Genie Code → the agent's **Instructions** → **add a SQL expression**; for each, give a **name**, paste the **SQL**, and mark it a *measure* or *filter*. Add:
- measure `readmission_rate` = `AVG(readmitted_30d) * 100`
- filter `inpatient_only` = `encounter_type = 'Inpatient'`

Now *"readmission rate by facility, inpatient only"* uses your governed definitions — same math every time, no re-deriving. *(Fuller set — `complication_rate`, `mortality_rate`, `avg_los` — is in `genie/genie_space_config.md`.)*
- **An example query** — teach one full, validated answer. Add the question *"30-day readmission rate by facility, min 200 encounters"* with this SQL:
`sql SELECT f.facility_name, ROUND(AVG(e.readmitted_30d)*100,1) AS readmit_rate_pct, COUNT(*) AS encounters FROM fact_encounters e JOIN dim_facility f USING (facility_id) GROUP BY f.facility_name HAVING COUNT(*) >= 200 ORDER BY readmit_rate_pct DESC`
Note the example spells out `AVG(readmitted_30d)*100` *rather than referencing the* `readmission_rate` *expression above. An example query must be complete, runnable SQL — a SQL expression is context Genie reads, not a column/function you can name in a query. Keep the two in sync (same math). This is the opposite of the metric views / SQL functions in Parts 2–5, which are real objects you call by name —* `MEASURE(...)` / `pcp_continuity_by_provider()`.

> **Priority order (what Genie reads, best first):** column comments/keys → **synonyms** → **SQL expressions** (metrics/filters) → **example queries** → free-text instructions *last*. The full set is in `genie/genie_space_config.md`.

Step 4 · Ask a few questions. Try these, then your own:

- "How many encounters were there in 2024 by region?" — `region` *comes from* `dim_facility`.
- "Which **doctors** have the highest 30-day readmission rate, with at least 200 encounters?" — *exercises your* `doctor` → `provider_name` *synonym.*
- "Which facilities have the highest 30-day readmission rate, with at least 200 encounters?" — *the* `dim_facility` *join + the example query you just added.*
- "What is the average length of stay for a {{PROCEDURE_EXAMPLE}}?"

Step 5 · Show the SQL. On any answer, click **Show generated code** to see the Databricks SQL Genie wrote — for the facility/region questions you'll see it joining `fact_encounters` to `dim_facility`.

> **Why this works — the setup drives the answer.** Genie's quality comes from the *setup*, not clever prompting. The context it reads is: **well-annotated tables** (every column commented), **explicit joins and keys**, and a **tight scope** (≤5 well-modeled tables). That's why the shared tables just work — curated, documented tables beat raw wide ones. *(This is the Databricks Day theme — "maturing Genie": engineer accuracy into the setup, then pin the known-good answers.)*
>
> **For the SQL folks:** Genie writes Databricks SQL, so it's a fast way to see the correct syntax and functions when you translate from what you write today. Copy it and build on it.
>
> **Trust:** every answer shows its SQL, Genie only sees tables you've been granted, and you can give feedback (Yes / Fix it / Request review). It's a fast first draft, not a black box.

Advanced features: governed metrics & trusted assets

Part 1 stood up a Genie Agent that gives *fast* answers in plain language. This half is the **accuracy layer**: making those answers *trustworthy*. It's the same arc as the Databricks Day session — the **confidence stack**, highest-confidence first:

1. **SQL functions** — deterministic, callable, audited logic.
2. **Metric views** — one governed definition of a metric, referenced by name everywhere.
3. **Trusted assets in Genie** — plain-language questions resolve to those governed definitions instead of ad-hoc SQL.

Everything runs on the **PCP continuity** use case, already built on the shared schema. *PCP continuity = the share of a patient's standard visits handled by their assigned PCP — a real quality measure with real business rules (standard visits only; attending must equal the assigned PCP). Those rules are exactly why it needs governing.* You'll **use** the pre-built assets first (Parts 2–4), then see how to **author your own** (Part 5).

Part 2 - SQL functions: deterministic, callable logic

> **Parts 2 and 3 are exploratory.** The function and metric view already exist in the shared schema — here you just *run queries against them* to see how governed assets behave. You'll build your own versions in **Part 5**.

A SQL function packages logic once, so anyone — including Genie — calls it **by name** instead of re-deriving joins. It's registered in Unity Catalog and governed (grant `EXECUTE`; callers can't see or change the logic inside — ideal for PHI-sensitive clinical rules). We use **one** function here — `pcp_continuity_by_provider()` — because it does the **provider grain the metric view can't** (the overall/facility/region ratio is covered by the metric view in Part 3). Run it in a SQL editor:

```
-- table function: continuity BY PROVIDER, min 200 standard visits
-- (the provider grain the metric view can't produce)
SELECT provider_name, standard_visits, ROUND(continuity_ratio*100,1) AS continuity_pct
FROM {{CATALOG}}.{{SCHEMA}}.pcp_continuity_by_provider()
WHERE standard_visits >= 200
ORDER BY continuity_ratio DESC;
```

It's a **table function** — you query it like a table, but the provider-grain logic lives in one governed place, not re-derived per query. Same inputs → same result every time (**deterministic**).

> **When do you reach for a function instead of the metric view?** When the question needs a **grain the metric view doesn't model**. The metric view (Part 3) rolls up by facility / region / month — it has **no provider dimension** — so *continuity by provider* is a job only the function can do. That's the clean division of labor: metric view for governed rollups, functions for the grains (and parameterized logic) it can't express.
>
> **A note on Genie + functions — this is a Part 4 concern, not something to set up here.** Right now (Part 2) you're just calling the function in a SQL editor, where it's fully deterministic. *Inside* Genie it behaves differently, and you'll deal with that when you build the agent in **Part 4**: Genie tends to prefer the metric view for anything it can answer that way, so adding the function (or a text instruction to use it) is **not** enough to make Genie call it. The fix, in Part 4, is to add an **example query** whose SQL calls the function (Configure → Examples → Add → Example Query) — and the **provider-grain question is where you test it**. Tested: pairing *"which providers have the lowest PCP continuity?"* to a `pcp_continuity_by_provider()` example query made Genie call the function; without it, Genie forced the question through the metric view (which has no provider dimension) and returned nothing.

Part 3 - Metric views: one metric, one number

A **metric view** defines a measure **once**, in Unity Catalog, so every dashboard, notebook, and Genie Agent computes it identically. `pcp_continuity_metrics` is already built. Query its measures with `MEASURE()` and group by its dimensions:

```
-- continuity by facility
SELECT `Facility`,
       ROUND(MEASURE(`PCP Continuity Ratio`)*100,1) AS continuity_pct,
       MEASURE(`Total Standard Visits`)             AS visits
FROM {{CATALOG}}.{{SCHEMA}}.pcp_continuity_metrics
GROUP BY `Facility`
ORDER BY continuity_pct DESC;
```

Now change **one line** — `GROUP BY `Region`` or `GROUP BY `Encounter Month`` — and the *same* measure re-slices correctly, no re-deriving:

```
SELECT `Region`, ROUND(MEASURE(`PCP Continuity Ratio`)*100,1) AS continuity_pct
FROM {{CATALOG}}.{{SCHEMA}}.pcp_continuity_metrics
GROUP BY `Region` ORDER BY continuity_pct DESC;
```

> **Why this matters (query-time grouping):** continuity is a **ratio** — it's *non-additive*, so averaging facility ratios does **not** give the regional ratio. A metric view resolves the grouping **at query time** from the underlying counts, so the number is mathematically correct at every grain. That's the trap a copy-pasted calculated field falls into — and the reason to define it once, centrally.

Part 4 - Trusted assets in Genie + the benchmark

Now wire the governed definitions into Genie so plain-language questions resolve to them.

First, see the problem — best-effort is phrasing-sensitive. On a *bare* agent (no trusted asset), ask the same thing two ways and click **Show generated code** each time:
- *"What is our overall PCP continuity rate?"* → Genie restricts to standard (office) visits → **76.3%** ✅
- *"What share of visits are with the patient's PCP?"* → Genie counts **all** visit types → **65.5%** ❌

Same question, an **11-point swing** by wording. Genie isn't "dumb" here — from the phrasing it inferred the standard-visits rule on the first one but not the second — and **nothing in the data forces it** (the standard-visits rule lives only in the governed assets, not as a flag on the tables). That's the trust problem: two analysts phrase it differently and report two different numbers.

Now anchor it to a governed definition — in two parts, because both matter:
1. **Add the governed assets to the agent.** Click **Configure**, go to the agent's **data sources**, click **Add**, and add the existing metric view `pcp_continuity_metrics` and the function `pcp_continuity_by_provider` (for provider-level questions). *(Adding*

them makes them available — but on its own, Genie often still hand-writes its own SQL. The routing instruction below is what makes it actually use them.)
2. **Point Genie at them with an instruction** (this is the step people skip). In **Instructions**, add a *routing* instruction — not the calculation itself:

> *For PCP continuity, use the governed assets rather than hand-writing SQL: query the* `pcp_continuity_metrics` *metric view with* `MEASURE()` *(e.g.* `MEASURE(\`PCP Continuity Ratio\`)` *), or call* `pcp_continuity_by_provider()` *for provider-level questions. They already encode the definition.*

Note what this instruction does not contain: the standard-visits/assigned-PCP rules. Those live in the metric view and function — restating them here would be redundant and would just hand Genie the recipe to hand-write the logic instead of calling the governed asset. Route to the asset; let the asset own the definition.

Now ask the continuity question, any phrasing, and **Show generated code**: Genie calls `MEASURE(\`PCP Continuity Ratio\`)` **by name** and returns **76.3%** — even for wordings that gave the naive agent 65.5%. *(In our testing, registering the asset alone left Genie hand-writing SQL; adding the instruction flipped it to call the metric view by name on every phrasing. The registration makes it available; the instruction makes it preferred.)*

One more lever — the provider question needs an example query. Steps 1–2 anchor the *rate* question to the metric view. The provider-grain question is different: Genie prefers the metric view and generally **won't call** `pcp_continuity_by_provider()` **on its own** — even for *"which providers have the lowest continuity?"*, which the metric view can't answer (it has no provider dimension). Registering the function and the routing instruction is **not** enough; give Genie an **example query**: click **Configure**, go to the agent's **Examples**, click **Add**, and select **Example Query**, then enter the question and paste the SQL:
- Question: *"Which providers have the lowest PCP continuity, with at least 200 standard visits?"*
- SQL:
`sql SELECT provider_name, standard_visits, ROUND(continuity_ratio*100,1) AS continuity_pct FROM {{CATALOG}}.{{SCHEMA}}.pcp_continuity_by_provider() WHERE standard_visits >= 200 ORDER BY continuity_ratio ASC`

Now ask that provider question and **Show generated code** — Genie calls the function. *(Tested: without the example query, Genie forced the question through the metric view and returned nothing.)* This is the escalation ladder: **register → instruct → example query**, escalating until Genie uses the asset — a function usually needs the example query; the metric view often just needs the instruction.

> **The honest mechanism** (matches the docs): Genie is **nondeterministic** even with trusted assets — it *decides* whether to call the function/query or just learn

> the rule from it. So the badge isn't guaranteed on every ask. What *is* reliable is the **single governed definition**: the metric view and function define continuity **once**, in Unity Catalog. Call them directly (SQL, dashboards) and you get the **same number every time, by definition** — that's the real determinism. In Genie they act as an **anchor** so answers converge on the governed number instead of drifting by phrasing like the naive agent. To *maximize* Genie using the asset (and showing the badge), also add it as a **parameterized example query** tied to a representative question.

Make it repeatable — the Benchmark and Monitor tabs. The two-phrasing test above is the idea done *by hand*. Two tabs at the top of the agent turn it into something you can measure and curate over time.

Benchmark — quantify the accuracy

A benchmark is a set of *question → expected-SQL* pairs. On **Run**, Genie answers each question its own way, executes both its SQL and your expected SQL, and scores whether they match — so you get a number, not a vibe.

Set it up:

1. Open your Genie agent and click **Benchmark** at the top of the agent.
2. Click **+ Add benchmark**, then enter the **question** and the **SQL**. Add all three below. *(Benchmarks 2 and 3 use the same expected SQL on purpose — two phrasings of "the rate," one governed answer; that's how you prove the phrasing drift is gone.)*

Benchmark 1 — facility ranking
- Question: `Which facilities have the lowest PCP continuity, and how many standard visits do they have?`
- SQL:

```
sql SELECT `Facility`, MEASURE(`PCP Continuity Ratio`) AS pcp_continuity_ratio, MEASURE(`Total Standard Visits`) AS total_standard_visits FROM {{CATALOG}}.{{SCHEMA}}.pcp_continuity_metrics GROUP BY `Facility` ORDER BY pcp_continuity_ratio ASC
```

> **Expect Benchmark 1 to score *Bad* — and that's the point.** "Which facilities have the **lowest**…" reads as a *top-N*, so Genie adds an `ORDER BY … LIMIT` and returns just the bottom few; your ground truth returns **all 12**. The scorer compares result sets exactly, so a different **shape** = Bad — even though every continuity value Genie returned is the **governed 76.3%-definition** number. This is a *shape/intent* mismatch, **not** a governance break: the metric is pinned, only the row count differs. The takeaway — benchmark scoring is strict exact-match, so your ground-truth SQL has to encode the answer *shape* you mean, and a fuzzy superlative like "lowest" forces you to decide what "the answer" is (all facilities

> ranked, or just the bottom N?). Benchmarks 2 and 3 below score **Good** — that's where you prove the phrasing drift on *the number* is gone.

Benchmark 2 — "share of visits with the PCP"
- Question: `What share of visits are with the patient's PCP?`
- SQL:
```
sql SELECT MEASURE(`Total Standard Visits`) AS total_standard_visits, MEASURE(`PCP Matched Visits`) AS pcp_matched_visits, MEASURE(`PCP Continuity Ratio`) AS pcp_continuity_ratio FROM {{CATALOG}}.{{SCHEMA}}.pcp_continuity_metrics GROUP BY ALL
```

Benchmark 3 — "overall continuity rate" *(same expected SQL as Benchmark 2)*
- Question: `What is the overall PCP continuity rate across all standard visits?`
- SQL:
```
sql SELECT MEASURE(`Total Standard Visits`) AS total_standard_visits, MEASURE(`PCP Matched Visits`) AS pcp_matched_visits, MEASURE(`PCP Continuity Ratio`) AS pcp_continuity_ratio FROM {{CATALOG}}.{{SCHEMA}}.pcp_continuity_metrics GROUP BY ALL
```

1. **Run** the benchmarks. Each row comes back with an **assessment**:
2. **Good** — Genie's generated SQL produced the same result as your expected SQL.
3. **Bad** — it differs; the row shows a **failure analysis** and a **SQL diff** — Genie's *Model output SQL* next to your *Ground truth SQL* — so you can see exactly what went wrong (a dropped filter, extra columns, wrong grain).

Monitor — curate from real usage

The **Monitor** tab turns real usage into improvements. It logs the **actual questions** analysts asked this agent, the SQL Genie generated for each, and any feedback left on them.

Walk it:

1. Open your Genie agent and select **Monitor** at the top.
2. You'll see the log of questions people have asked — each row shows the **question**, the **SQL Genie generated**, and the answer, plus any 👍 / 👎 / *Fix it* feedback.
3. Scan for the misses — 👎 'd answers, or rows where the generated SQL is wrong. Each miss is a curation task: promote it into the right governed context —
4. an everyday word Genie didn't map → add a **synonym**
5. a metric it recomputed by hand → point it at the **metric view** (or add a **SQL expression**)
6. a question it should route to the function but didn't → add an **example query** (the lever from earlier in Part 4)

7. missing governed logic → register the **trusted asset** + a routing instruction
8. Close the loop with the **Benchmark** tab: add the fixed question as a benchmark row and **Run**, so the fix is proven and stays fixed.

> **Seed it before you demo.** The **Monitor** tab only fills from real asks, so it's empty on a brand-new agent. Ask the agent a few questions first — the two rate phrasings and the provider question from above — so the tab has content to show. Ask the naive *"what share of visits are with the patient's PCP?"* (the **65.5%** miss) and 👎 it: that's the exact row you then fix and re-benchmark, live.
>
> **The loop: Monitor** tells you *what to fix* → a trusted asset / instruction / example query *fixes it* → the **Benchmark** *proves it stayed fixed*. That's how a Genie Agent matures from "demo" to "trusted."

Part 5 - Author your own metric view & function *(extra exercise, if time)*

You've *used* the governed assets in Parts 2–4 — now **build them yourself** to get hands-on with authoring metric views and SQL functions. You'll re-create the two you saw: the `pcp_continuity_metrics` metric view and the `pcp_continuity_by_provider` function. Build them in `{{CATALOG}}.analyst101_<you>` (your own schema), reading the shared **synthetic** base data you have SELECT on. Same steps, same syntax you'll use to stand up metric views and functions on your **own** data back home — this is the reps, on safe data.

> **Don't have your own schema yet?** Create one first — left nav → **Catalog** → click the catalog `{{CATALOG}}` → **Create schema** (top-right) → name it `analyst101_<you>` (e.g. `analyst101_jsmith`), leave the default storage location, **Create**. *(If Create schema is greyed out, tell your instructor — the workshop group needs `CREATE SCHEMA` on `{{CATALOG}}`.)*

1 · The metric view — one governed definition. A metric view's `source` is a single relation, so continuity sources the shared `encounters_enriched` convenience view (one row per encounter with `visit_type_id` and an `is_pcp_match` flag already computed; standard visits are `visit_type_id = 'OFFICE'`). Run this in a SQL editor:

```
CREATE OR REPLACE VIEW {{CATALOG}}.analyst101_<you>.pcp_continuity_metrics
(`Facility`, `Facility Code`, `Region`, `Encounter Month`,
 `Total Standard Visits`, `PCP Matched Visits`, `PCP Continuity Ratio`)
COMMENT 'Governed PCP continuity metrics over STANDARD visits only. Define the ratio once
WITH METRICS
LANGUAGE YAML
AS $yaml$
version: 0.1
source: {{CATALOG}}.{{SCHEMA}}.encounters_enriched
filter: visit_type_id = 'OFFICE'
dimensions:
  - name: Facility
    expr: facility_name
  - name: Facility Code
    expr: facility_id
  - name: Region
    expr: region
  - name: Encounter Month
    expr: date_trunc('MONTH', admit_date)
measures:
  - name: Total Standard Visits
    expr: COUNT(1)
  - name: PCP Matched Visits
    expr: COUNT_IF(is_pcp_match)
  - name: PCP Continuity Ratio
    expr: TRY_DIVIDE(COUNT_IF(is_pcp_match), COUNT(1))
$yaml$;
```

Query it like the shared one — measures via `MEASURE()`, grouped by a dimension:

```
SELECT `Facility`, ROUND(MEASURE(`PCP Continuity Ratio`)*100,1) AS continuity_pct
FROM {{CATALOG}}.analyst101_<you>.pcp_continuity_metrics
GROUP BY `Facility` ORDER BY continuity_pct DESC;
```

(Prefer the UI? Catalog Explorer → Create → Metric view offers AI-assisted authoring over the same YAML.)

2 · The `pcp_continuity_by_provider` function — the provider grain the metric view can't do. A table function, self-contained (it joins the base tables in its body), parameterized by a date window:

```
CREATE OR REPLACE FUNCTION {{CATALOG}}.analyst101_<you>.pcp_continuity_by_provider(
  p_start STRING DEFAULT '2023-01-01',
  p_end   STRING DEFAULT '2025-12-31')
RETURNS TABLE (provider_name STRING, standard_visits BIGINT, matched_visits BIGINT, conti
COMMENT 'PCP continuity ratio per attending provider over standard visits in the date win
RETURN
  SELECT pr.provider_name,
         count(*) AS standard_visits,
         count_if(e.provider_id = pt.assigned_pcp_id) AS matched_visits,
         try_divide(count_if(e.provider_id = pt.assigned_pcp_id), count(*)) AS continuity
  FROM {{CATALOG}}.{{SCHEMA}}.fact_encounters e
  JOIN {{CATALOG}}.{{SCHEMA}}.dim_patient    pt ON e.patient_id    = pt.patient_id
  JOIN {{CATALOG}}.{{SCHEMA}}.dim_provider   pr ON e.provider_id   = pr.provider_id
  WHERE e.visit_type_id = 'OFFICE' AND e.admit_date BETWEEN to_date(p_start) AND to_date(
  GROUP BY pr.provider_name;
```

Call it (parameters have defaults, so `()` works):

```
SELECT provider_name, standard_visits, ROUND(continuity_ratio*100,1) AS continuity_pct
FROM {{CATALOG}}.analyst101_<you>.pcp_continuity_by_provider()
WHERE standard_visits >= 200 ORDER BY continuity_ratio DESC;
```

Grant `EXECUTE` to your Genie users, register these as trusted assets, and add an example query so Genie calls the function (Part 2's note) — and you've rebuilt the whole confidence stack on your own. *(The full reference DDL, including the* `encounters_enriched` *view, is in* `advanced_module/continuity_assets.sql`*.)*

Capstone (take-home) — apply this to your own data

This is your **leave-behind**. The workshop used synthetic data so you could learn the moves safely; the capstone is where you point those same moves at **your own business data**. Use it back at your desk — pick a real question your team asks and build the answer here: a dashboard, a Genie Agent, or a governed metric.

Start by thinking about your data:
- What are your core **fact** and **dimension** tables (the things you count, and the things you slice by)?
- What's the one **metric** whose definition people argue about? That's your first governed metric view.
- Which **Tableau dashboard** would you most want to rebuild — and what would "better" look like?

- What's a question people ask in Slack/email that a **Genie Agent** on your data could just answer?

Scenario ideas — pick one, or bring your own (all on *your* data):

Scenario	What it practices	On your data, think about…
A team / entity scorecard	aggregation, ranking, thresholds	your KPIs by rep, store, product, or region — rank them and threshold the ones that matter
A deep-dive on one segment	filtering, trend over time	pick one product line, customer segment, or cohort and trend its key measures
Two measures against each other	scatter, correlation	cost vs. outcome, spend vs. revenue, effort vs. result — one point per entity
A categorical mix	stacked bar, share-of	your mix by channel, category, or segment across a dimension
Genie-first exploration	NL querying → save to dashboard	ask your governed data plain-language questions, then pin the good answers
A governed metric	metric view + trusted asset	define your most-argued-about metric **once**, so every dashboard and Genie answer matches
Rebuild a Tableau dashboard	direct comparison	recreate one you know well; note what's easier and what's missing

Techniques to work in (whatever your scenario):

- A filter and cross-filtering between two widgets.
- A drill-down hierarchy.
- A Genie question with the SQL revealed.
- A governed **metric view** for your headline number (the Part 5 pattern, on your data).

Before you leave, jot down your first move back home: the dashboard or Genie Agent you'll build, the tables it needs, and the one metric worth governing first.